

Managing Memory 2

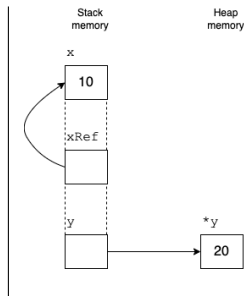
Manipulating Strings with Pointers

Department of Computer Science
University of Pretoria

9 May 2024

Recap

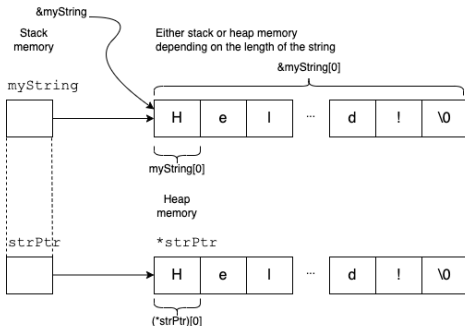
```
int x = 10;           // on the stack  
int &xRef = x;        // reference to x  
int *y = 20;         // on the heap
```



- A `string` is class defined in the `string` library and forms part of the `std` namespace.
- An example of a string definition is given by:
`string myString = "Hello World!";`
- `myString` is stored on the stack.
- The continuous characters “Hello World!” of `myString` are *typically* stored on the stack unless the string contains more than ± 14 characters, then it is stored on the heap.

- What about the following string?
`string *strPtr = new string("Hello, world!");`
- The continuous characters "Hello World!" are stored on the heap.
- Our handle (`strPtr`) that will enable us to gain access to the string, is on the stack

- Strings can be accessed using their reference/pointer
- Strings are indexed from zero from where the reference/pointer begins
- All characters from the reference/pointer are part of the string until the terminating string character ('`\0`') is encountered.



C++ Statement

```
string myString = "Hello World!";
```

```
string *strPtr = new string("Hello, world!");
```

- Never increment or decrement the reference/pointer that was used when creating the string. You WILL lose your access to the beginning of the string.
- Always assign the first character of the string and in some circumstances the last character of the string to a newly defined pointer variable, specifically a variable of `char*`.
- For a string defined as: `string myString`, the code will be:
`char *c = &myString[0];`
- For a string defined as: `string *strPtr = new string("Hello, world!");`, you will need to convert the string object to a c-string using the following code:
`const char *d = strPtr->c_str();`

- string has a predefined function to calculate the length of a string.
- The code below shows how to call the built in length function for the two string definitions given previously.

```
string myString = (" Hello World");  
string *strPtr = new string(" Hello , world!");  
  
cout << "using myString to call the length function: "  
      << myString.length() << endl;  
cout << "using strPtr to call the length function: "  
      << strPtr->length() << endl;
```

- Now write your own function, that does not make use of the built-in length function, to determine the length of a string.
- The definition for your own function is given below along with how the function is called for each of the defined strings.

```
int  stringLength(string &str); // Function definition

string myString = (" Hello World");
string *strPtr = new string(" Hello , world!");

cout << stringLength(myString) << endl;
cout << stringLength(*strPtr) << end;
```



```
int stringLength(string &str) {  
    int len = 0;  
    char *current;  
    current = &str[0];  
    while (*current != '\0') {  
        len++;  
        current++;  
    }  
    return len;  
}
```

Apply algorithmic thinking to the problem of designing an algorithm to reverse a given string.

- Design a function to swap two character values.
- Design a function to reverse a string using pointers to chars to step through the string. Make use of your swap function in your solution.
- What are the function definitions for the swap and reverse functions respectively?
- Translate your solution to C++ code and thoroughly test your code.